

P8: Another Language

Name: Alston Liow Guan You

Student ID: 37440985

Date: 23 August 2026

Overview

This task is an opportunity to explore another language: Python.

Tip: Foundations

This is a test of the foundations you have established. The better you have understood the things we have covered so far, the easier this task will be. Use this as a chance to test yourself: are you able to see how these are similar and different? Python handles more for you, can you see what it is doing? Can you use what you have learnt to see how the system underpinning Python is working?

Do the following:

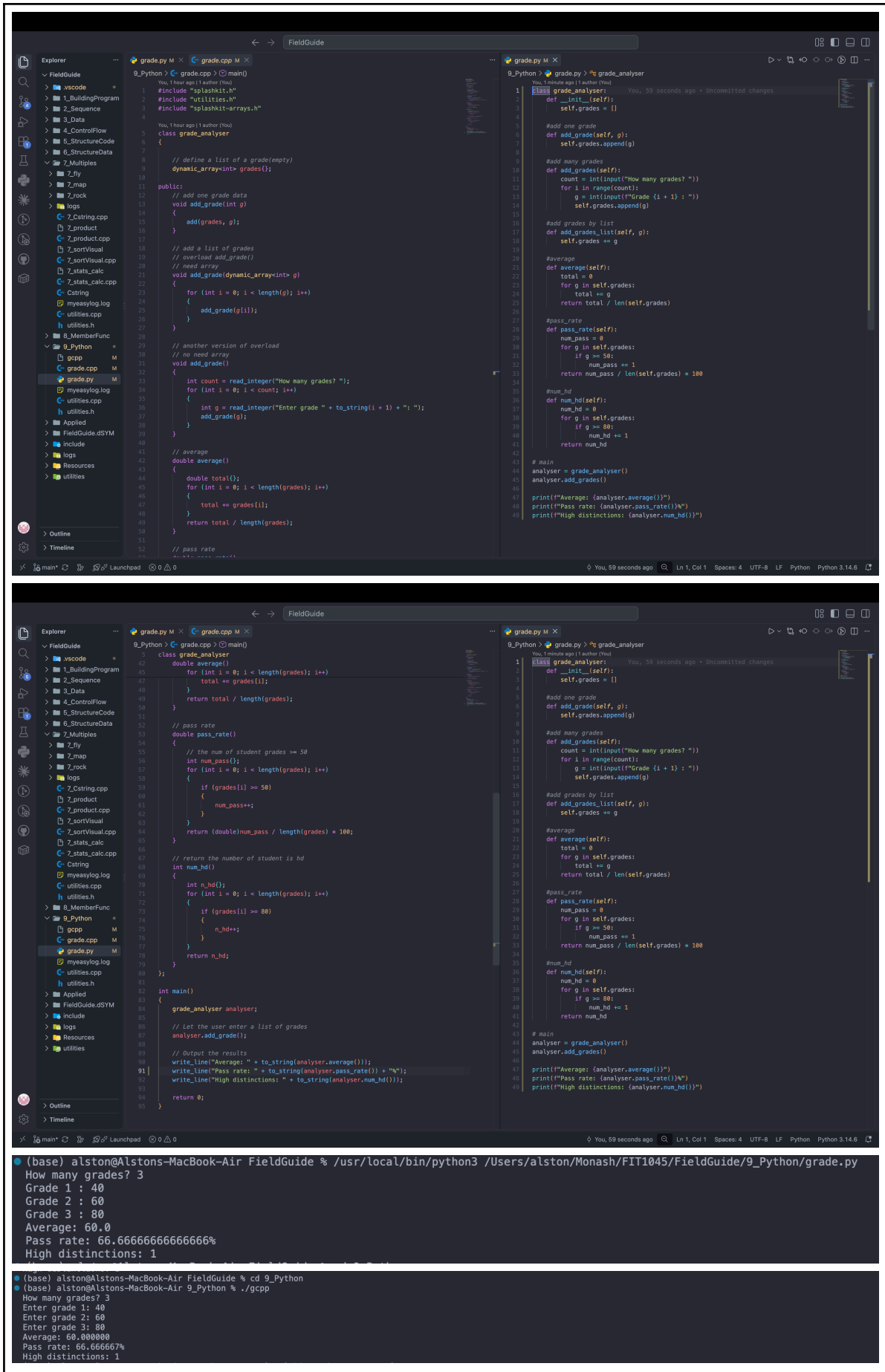
- Ensure you have a working version of [Python](#) installed on your computer.
- Identify and use resources to help you get started with Python. We recommend:
 - [The Python Tutorial](#) - this is the official Python documentation. I suggest skimming (or skipping) [Page 1](#) and [Page 2](#). [Page 3](#) provides a good introduction to expressions, statements, and the main types - work through this. Similarly, [Page 4](#) covers control flow and functions, so use this to get started with the various control flow statements and to build your own functions. [Page 6](#) shows how to import modules - like `using` in C# or `#include` in C/C++. Note the section on Page 6 about [Executing modules as scripts](#) - this shows how to create an equivalent of a main function by checking if this module is called “__main__”.
 - If you need other support, consider some of the online tutorials and books such as [Foundations of Python Programming](#) from Runestone academy, or [Introduction to Python](#) from Geeks for Geeks. These cover the concepts alongside the syntax, so you can align the concepts with what you already know.
- Add a screenshot to show that you can map what you have learnt from C/C++ to Python. Create a C/C++ and Python version of one of the following programs, or a similar program of your own invention. Each program needs to involve the creation of functions, use of control flow, arrays/lists, and user input/output. Put your C/C++ and Python versions side by side.
 - Temperature Tracker
Asks the user to input a number of daily temperatures (array/list of doubles) then output the following details:
 - Calculate and display the average temperature.
 - Display the highest and lowest temperatures.
 - Count how many days were above the average.
 - Grocery List Manager

Allow the user to build a grocery list (array/list of strings). The user should be able to:

- Add items to the list
- Remove items from the list
- View the list

- **Student Grade Analyser**

Allow the user to enter a list of grade results (array/list of integers). Output the average grade, pass rate (% that are $\geq 50/100$) and number of high distinction achievements (number $\geq 80/100$).



The screenshot shows a code editor with two tabs: `grade.cpp` and `grade_analyser.py`.

grade.cpp (C++ code):

```

1 #include "splashkit.h"
2 #include "utilities.h"
3 #include "splashkit-arrays.h"
4
5 // You, 1 hour ago (1 author) (You)
6
7 // define a list of a grade(empty)
8 dynamic_array<int> grades();
9
10
11 public:
12 // add one grade data
13 void add_grade(int g)
14 {
15     add(grades, g);
16 }
17
18 // add a list of grades
19 // overload add_grade()
20 // need array
21 void add_grade(dynamic_array<int> g)
22 {
23     for (int i = 0; i < length(g); i++)
24     {
25         add_grade(g[i]);
26     }
27 }
28
29 // another version of overload
30 // no need array
31 void add_grade()
32 {
33     int count = read_integer("How many grades? ");
34     for (int i = 0; i < count; i++)
35     {
36         int g = read_integer("Enter grade " + to_string(i + 1) + ": ");
37         add_grade(g);
38     }
39 }
40
41 // average
42 double average()
43 {
44     double total();
45     for (int i = 0; i < length(grades); i++)
46     {
47         total += grades[i];
48     }
49     return total / length(grades);
50 }
51
52 // pass rate
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95

```

grade_analyser.py (Python code):

```

1 class grade_analyser:
2     def __init__(self):
3         self.grades = []
4
5     # add one grade
6     def add_grade(self, g):
7         self.grades.append(g)
8
9     # add many grades
10    def add_grades(self):
11        count = int(input("How many grades? "))
12        for i in range(count):
13            g = int(input("Grade (" + str(i + 1) + "): "))
14            self.grades.append(g)
15
16    # add grades by list
17    def add_grades_list(self, g):
18        self.grades += g
19
20    # average
21    def average(self):
22        total = 0
23        for g in self.grades:
24            total += g
25        return total / len(self.grades)
26
27    # pass_rate
28    def pass_rate(self):
29        num_pass = 0
30        for g in self.grades:
31            if g >= 50:
32                num_pass += 1
33        return num_pass / len(self.grades) * 100
34
35    # num_hd
36    def num_hd(self):
37        num_hd = 0
38        for g in self.grades:
39            if g >= 80:
40                num_hd += 1
41        return num_hd
42
43    # main
44    analyser = grade_analyser()
45    analyser.add_grades()
46
47    print("Average: " + str(analyser.average()))
48    print("Pass rate: " + str(analyser.pass_rate()) + "%")
49    print("High distinctions: " + str(analyser.num_hd()))

```

Terminal Output:

```

(base) alston@Alstons-MacBook-Air FieldGuide % /usr/local/bin/python3 /Users/alston/Monash/FIT1045/FieldGuide/9_Python/grade.py
How many grades? 3
Grade 1: 40
Grade 2: 60
Grade 3: 80
Average: 60.0
Pass rate: 66.66666666666666%
High distinctions: 1

```

```

(base) alston@Alstons-MacBook-Air FieldGuide % cd 9_Python
(base) alston@Alstons-MacBook-Air 9_Python % ./gccpp
How many grades? 3
Enter grade 1: 40
Enter grade 2: 60
Enter grade 3: 80
Average: 60.000000
Pass rate: 66.666667%
High distinctions: 1

```

> All concept of building a program is same. The input and out put is same, just different name. The logic of building a function is same. Control flow and sequence is same, just different way to write.

> C++ splash kit lib need to use `dynamic_array` for a array, but Python just need list `[]`. Python can loop a list easily using `for n in range`, C++ more complex. C++ need `int main()` to put a main program and always using `{, ";`, python no need just write anyway and using tab, `:"`. C++ can over load, python cannot. C++ need declare the type of data, python no need. Python need self in class.

Reflect:

- What was the most interesting thing you learnt through this task, and why?

> I learned about how to building a program with difference language.

- Was Python easy/hard to pick up? How did your understanding of the foundations of programming help?

> Python is very easy to pick up. I already know how to build a program, so when I learned a new language, just need to learn about the syntax.

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> I use 2 hours to complete this task. I spend time to build grade system with two different language.

Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I did not use any AI for this task.

Submit:

- Export this worksheet as a PDF.
- Upload your submission to OnTrack: